usage. `try_steal()` takes items from the opposite end of the queue to `try_pop()` in order to minimize contention; you could potentially use the techniques discussed in chapters 6 and 7 to enable concurrent calls to `try_pop()` and `try_steal()`.

OK, so you have your nice sparkly work queue that permits stealing; how do you use it in your thread pool? Here's one potential implementation.

Listing 9.8. A thread pool that uses work stealing

```
class thread_pool
{
    typedef function_wrapper task_type;

    std::atomic_bool done;
    thread_safe_queue<task_type> pool_work_queue;
    std::vector<std::unique_ptr<work_stealing_queue> > queues;     <-1
    std::vector<std::thread> threads;
    join_threads joiner;

    static thread_local work_stealing_queue* local_work_queue;     <-2
    static thread_local unsigned my_index;

    void worker_thread(unsigned my_index_)
    {
        my_index=my_index_;
        local_work_queue=queues[my_index].get();     <-3
        while(!done)
        {
            run_pending_task();
        }
    }
```